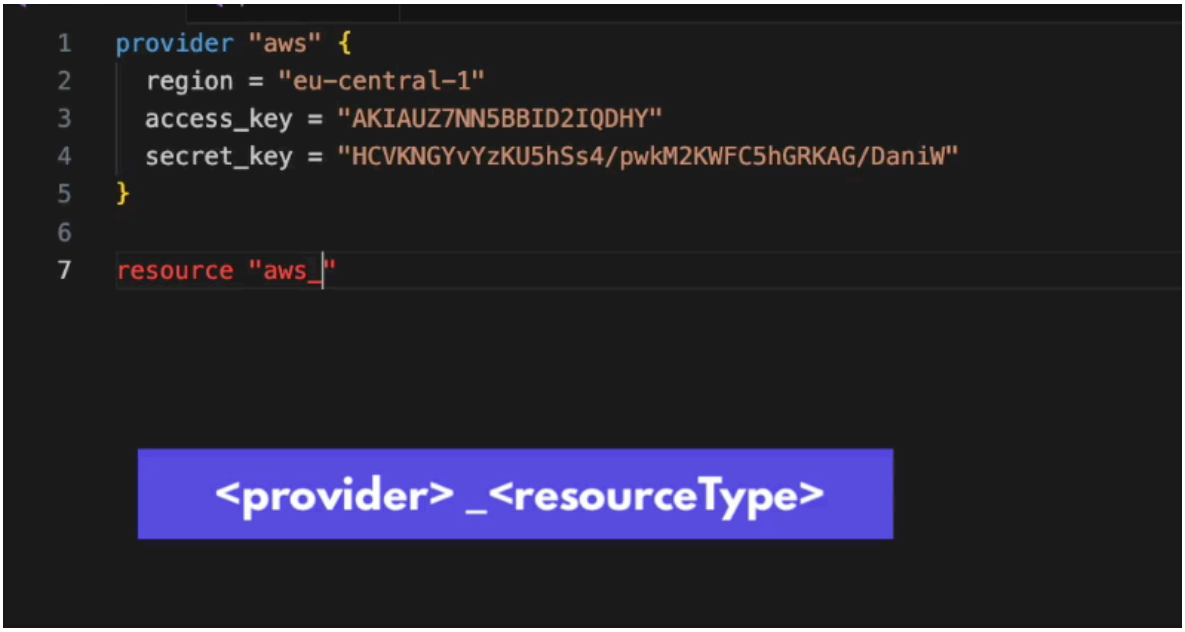
Module 12 - chapter 4
Resources & Data Sources

Resources

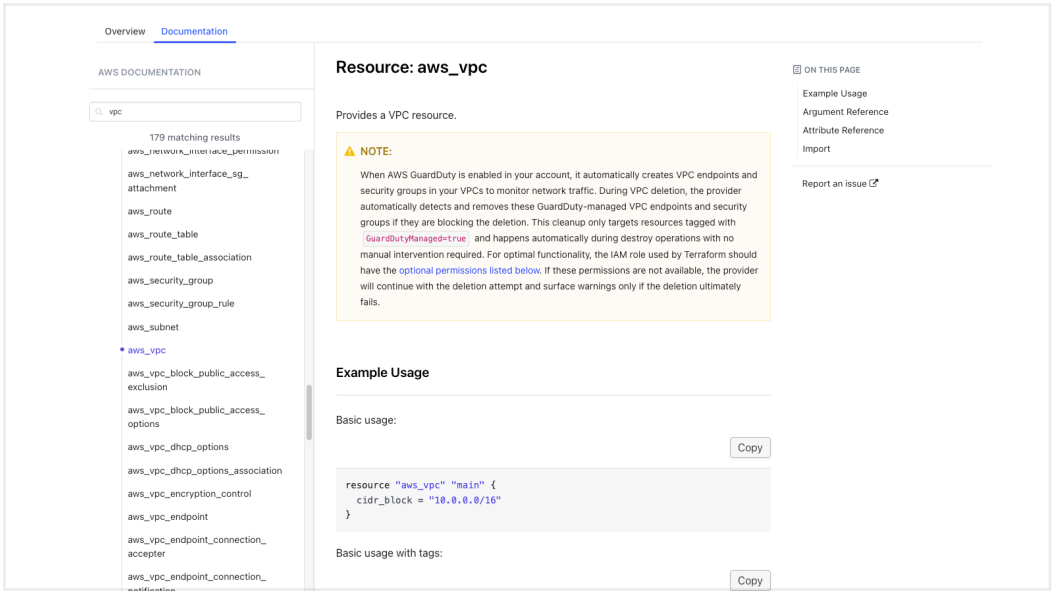
We use keyword -> resource

And we need to know the name of the resource, for example we want to create a VPC

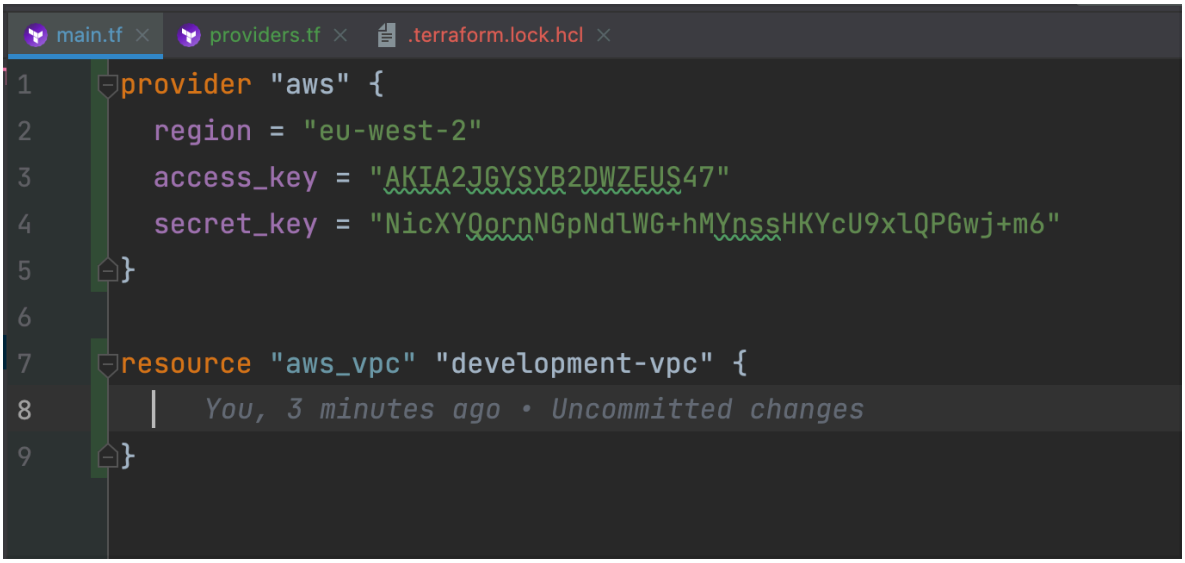
<provider>_<resourceTYpe>



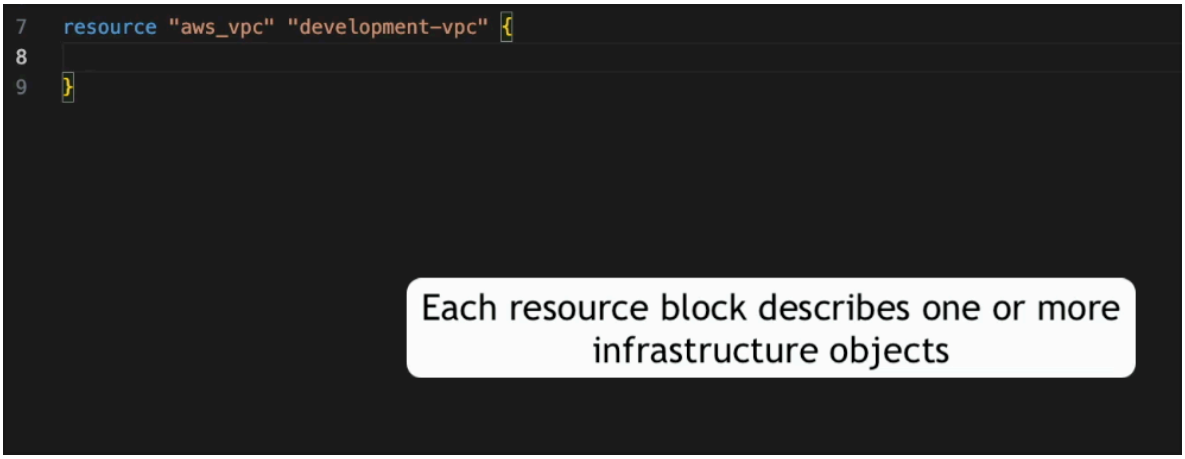
name of the resource -> “aws_vpc”



Name inside terraform, a **variable name** that we define -> “development-vpc” see line 7 below:



And inside the block we can pass in parameters

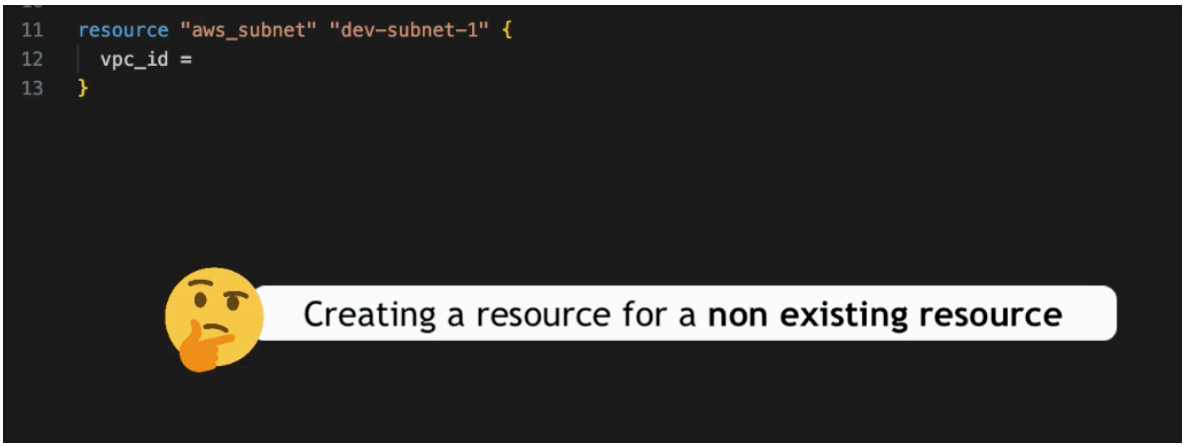


VPC needs an infrastructure object called CIDR block to define the IP address range for our VPC:

Let’s add another resource, a subnet:

```
7   resource "aws_vpc" "development-vpc" {
8     cidr_block = "10.0.0.0/16"
9   }
10
11  resource "aws_subnet" "dev-subnet-1" {
12
13  }   You, Moments ago • Uncommitted changes
```

Now a subnet is part of a VPC so we need to define the VPC that the subnet will be part of. And we want the subnet to be part of VPC “development-vpc“ but this VPC resource doesn’t exist just yet:



So we reference the resource to get the object-> resource name.varName

-> aws_vpc.development-vpc

And now from the object we can access the object’s attributes like ID:

So we have referenced a resource that hasn't been created yet.

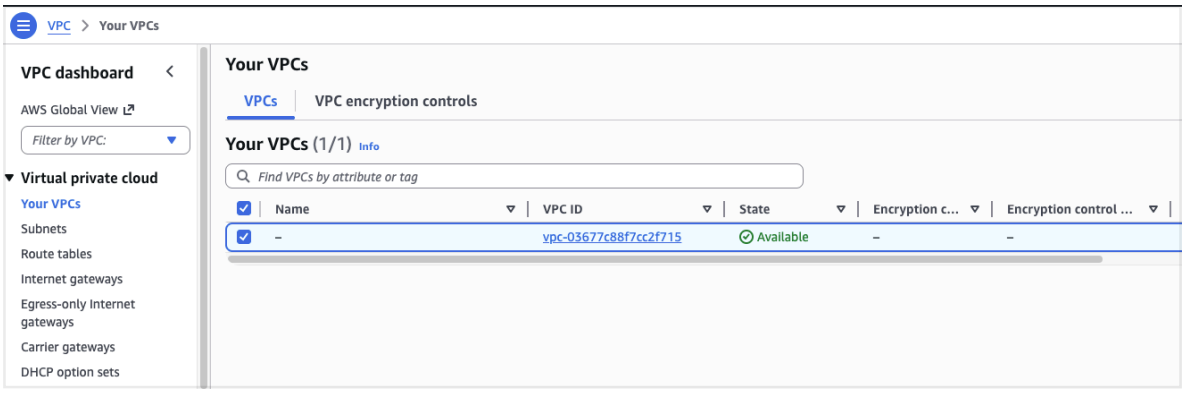
We can now define the CIDR block for the subnet:

And we define the AZ the subnet will be created in, at the moment it will used a default AZ but we want to specify it:

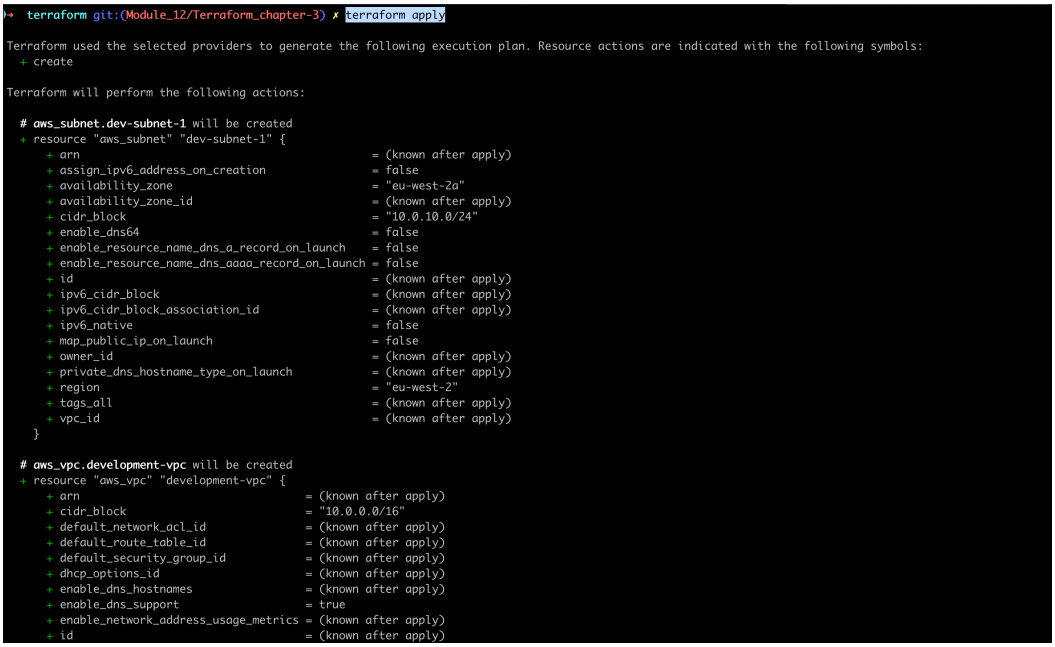
Terraform apply

terraform apply —> reads your .tf files, figures out what needs to be created/changed/deleted in AWS, shows you a plan, and after you confirm it executes the changes.

Currently my AWS has only one VPC

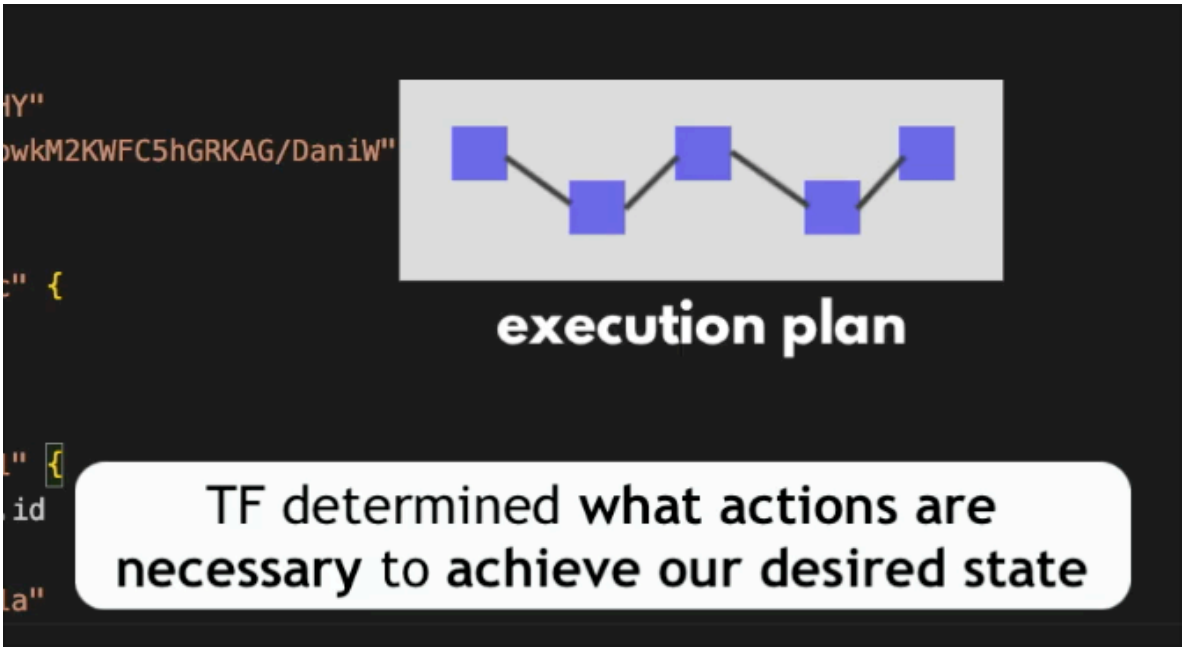


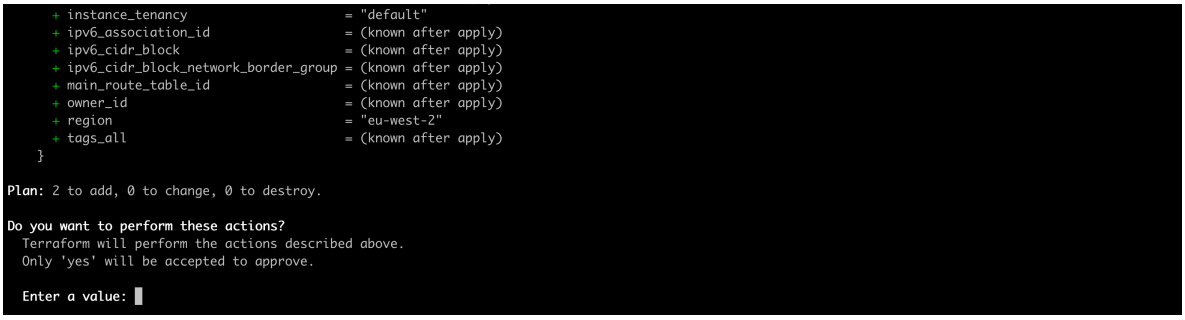
After I execute terraform apply I will have two VPCs, one of which was created by terraform
-> terraform apply



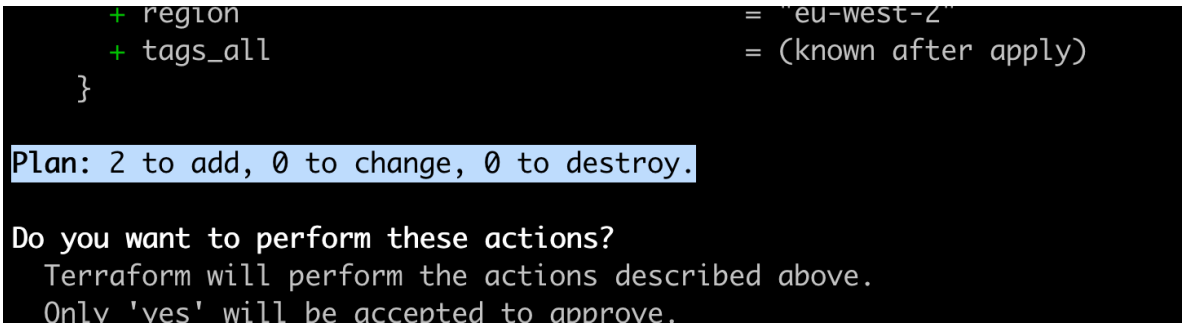
```
→  terraform git:(Module_12/Terraform_chapter-3) ✗ terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_subnet.dev-subnet-1 will be created
  + resource "aws_subnet" "dev-subnet-1" {
      + arn                                            = (known after apply)
      + assign_ipv6_address_on_creation                = false
      + availability_zone                              = "eu-west-2a"
      + availability_zone_id                           = (known after apply)
      + cidr_block                                     = "10.0.10.0/24"
      + enable_dns64                                   = false
      + enable_resource_name_dns_a_record_on_launch    = false
      + enable_resource_name_dns_aaaa_record_on_launch = false
      + id                                             = (known after apply)
      + ipv6_cidr_block                                = (known after apply)
      + ipv6_cidr_block_association_id                 = (known after apply)
      + ipv6_native                                    = false
      + map_public_ip_on_launch                        = false
      + owner_id                                       = (known after apply)
      + private_dns_hostname_type_on_launch            = (known after apply)
      + region                                         = "eu-west-2"
      + tags_all                                       = (known after apply)
      + vpc_id                                         = (known after apply)
    }

  # aws_vpc.development-vpc will be created
  + resource "aws_vpc" "development-vpc" {
      + arn                                  = (known after apply)
      + cidr_block                           = "10.0.0.0/16"
      + default_network_acl_id               = (known after apply)
      + default_route_table_id               = (known after apply)
      + default_security_group_id            = (known after apply)
      + dhcp_options_id                      = (known after apply)
      + enable_dns_hostnames                 = (known after apply)
      + enable_dns_support                   = true
      + enable_network_address_usage_metrics = (known after apply)
      + id                                   = (known after apply)
```

Terraform is showing us the plan, the actions that need to be taken to achieve our desired state 👆

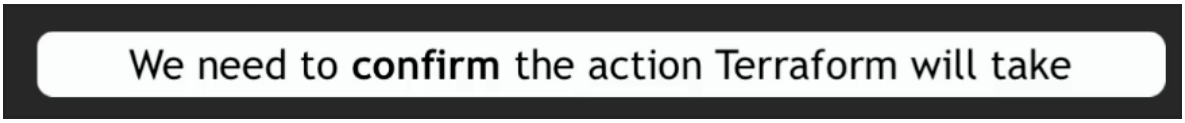




We also can see the summary



And we have to provide a value



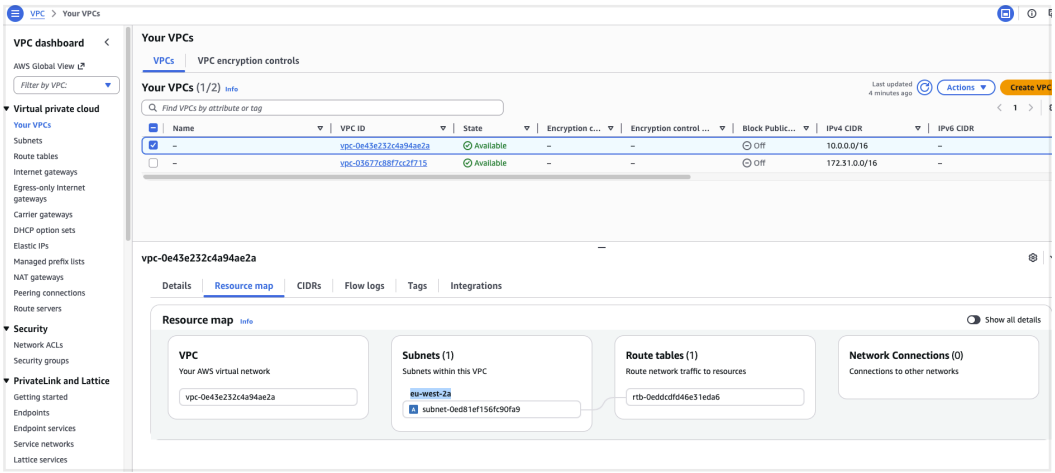
```
      + region                                 = "eu-west-2"
      + tags_all                               = (known after apply)
    }

Plan: 2 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value:
```

If after reviewing the information everything looks as we want to, then we type ‘yes’ any other string will result in canoeing the ‘terraform apply’

-> yes

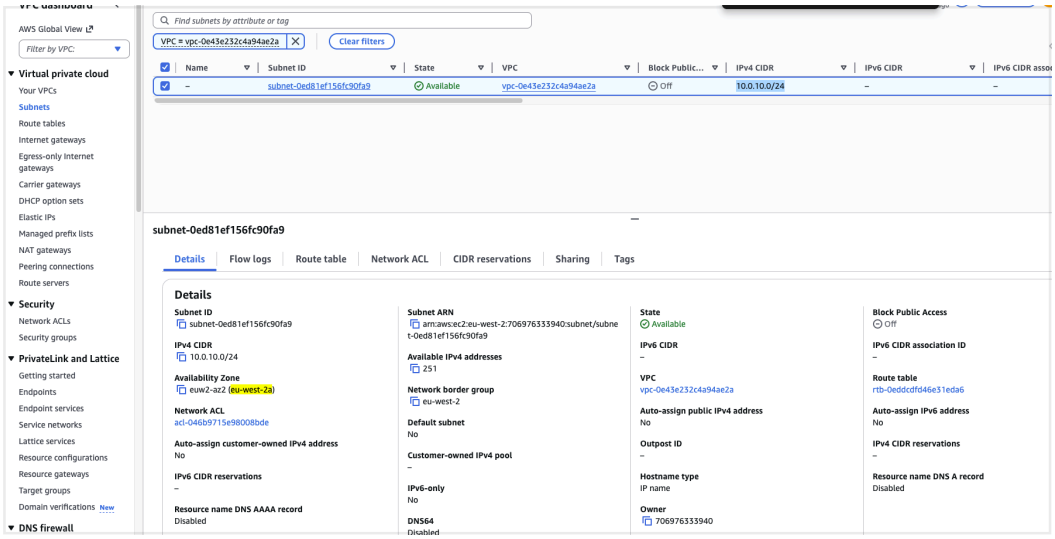
```
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_vpc.development-vpc: Creating...
aws_vpc.development-vpc: Creation complete after 2s [id=vpc-0e43e232c4a94ae2a]
aws_subnet.dev-subnet-1: Creating...
aws_subnet.dev-subnet-1: Creation complete after 0s [id=subnet-0ed81ef156fc90fa9]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
➜  terraform git:(Module_12/Terraform_chapter-3) ✗
```

And in my AWS management console I can see my new VPC:



The VPC doesn’t have a name because we didn’t set a tag in tf for the vpc.

And here is the new subnet

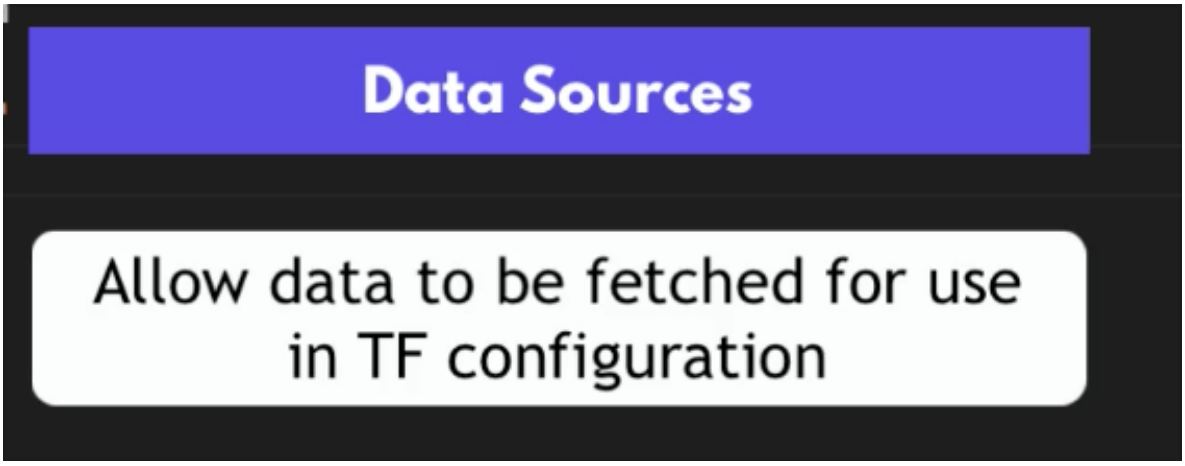


The resources were applied successfully 🥳

Data Sources

What if I want to create a subnet for an existing VPC?
I will need the id of the exiting VPC.

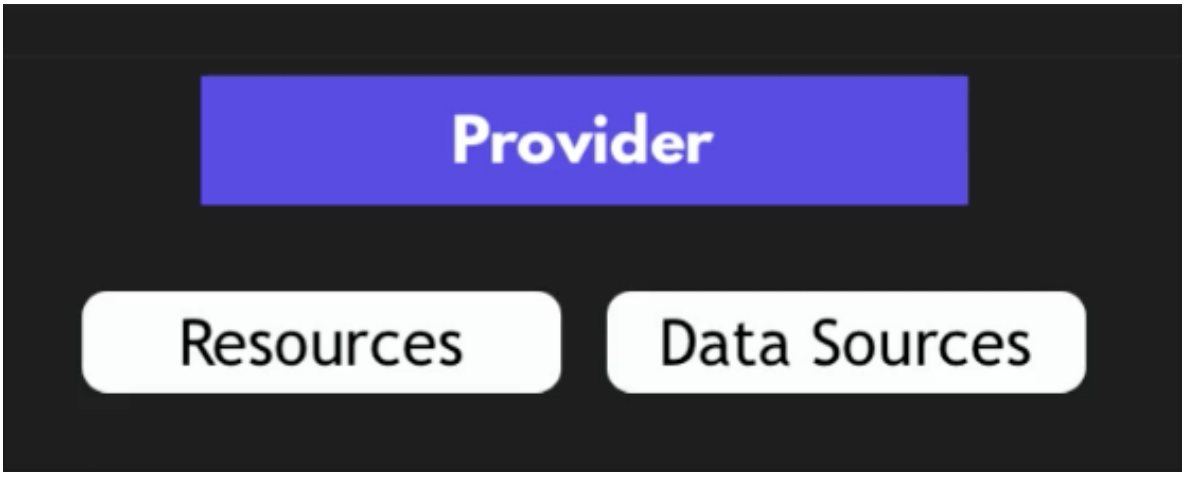
To get the VPC's id I can use the UI but is not efficient, so an alternative is to query the information from AWS using the provider



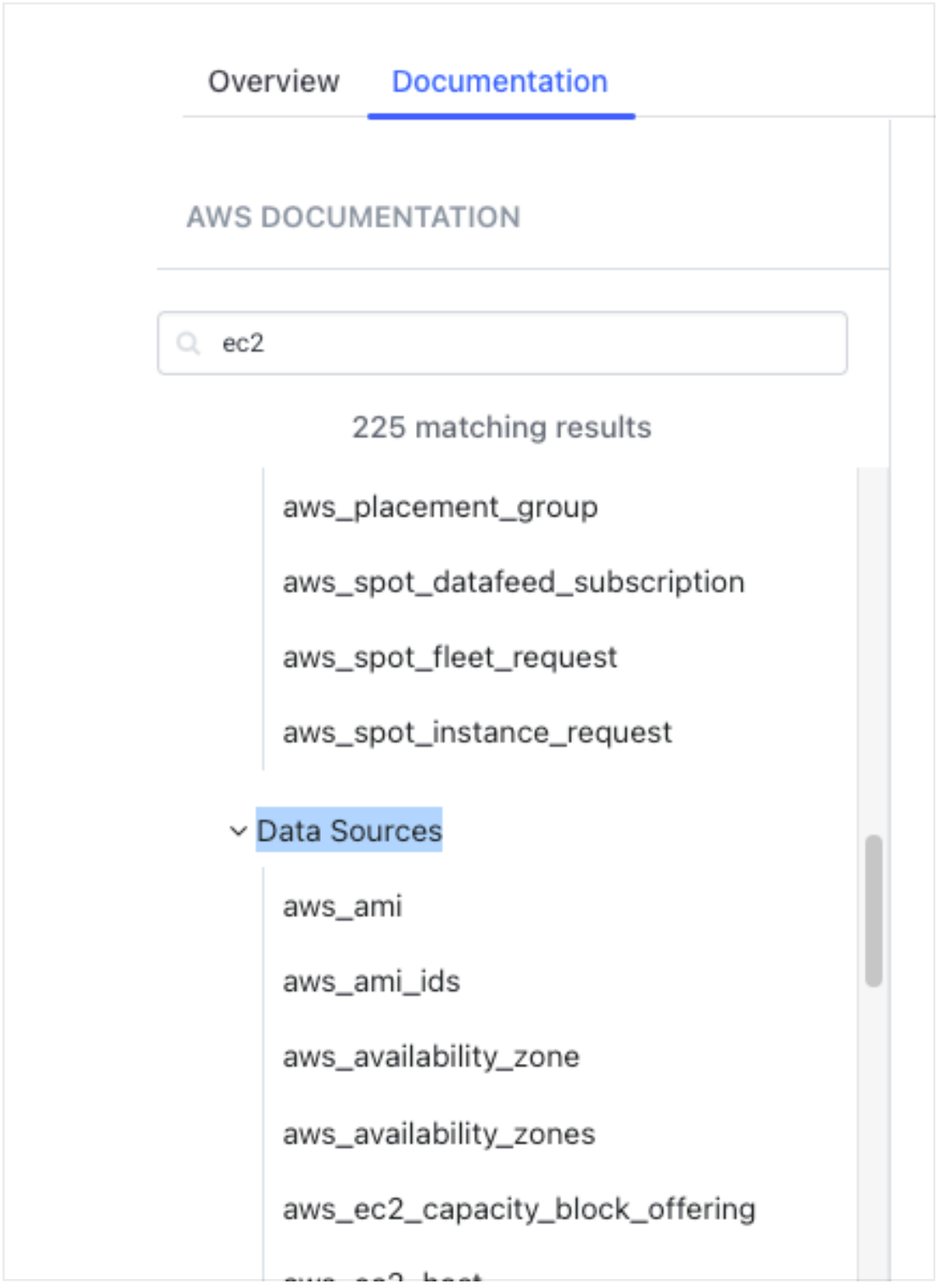
And the component that the provider give us to query the information is 'data'.

Data -> lets us query the existing resources and components from AWS ('resource' let's us create new resources)

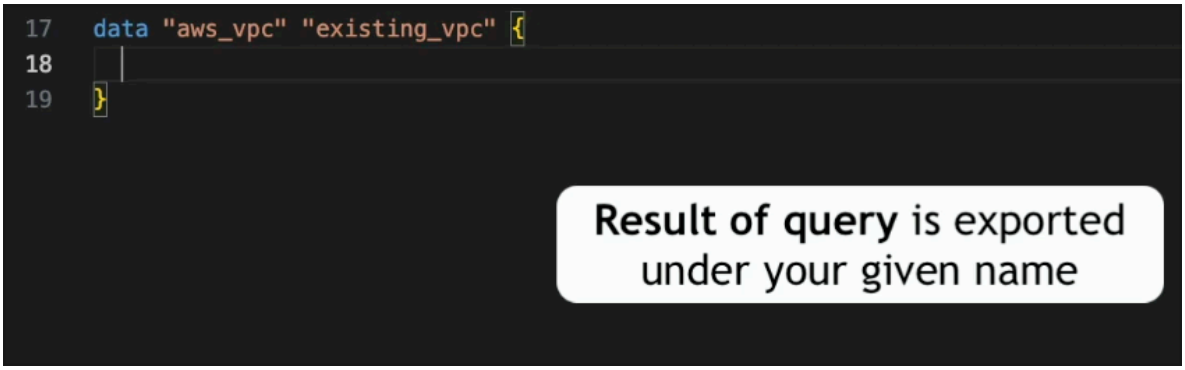
Two types of components that we get from the provider:



We can see the data sources (along with the resources) in the documentation, for example see below the the data sources fro EC2:

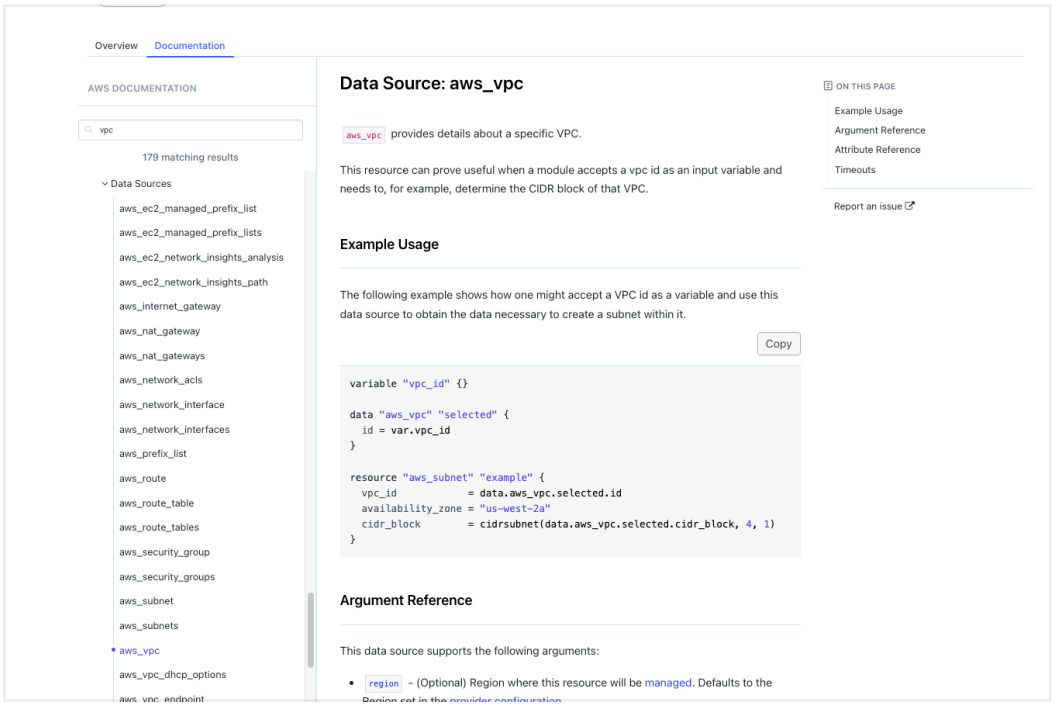


Now let's add a query:

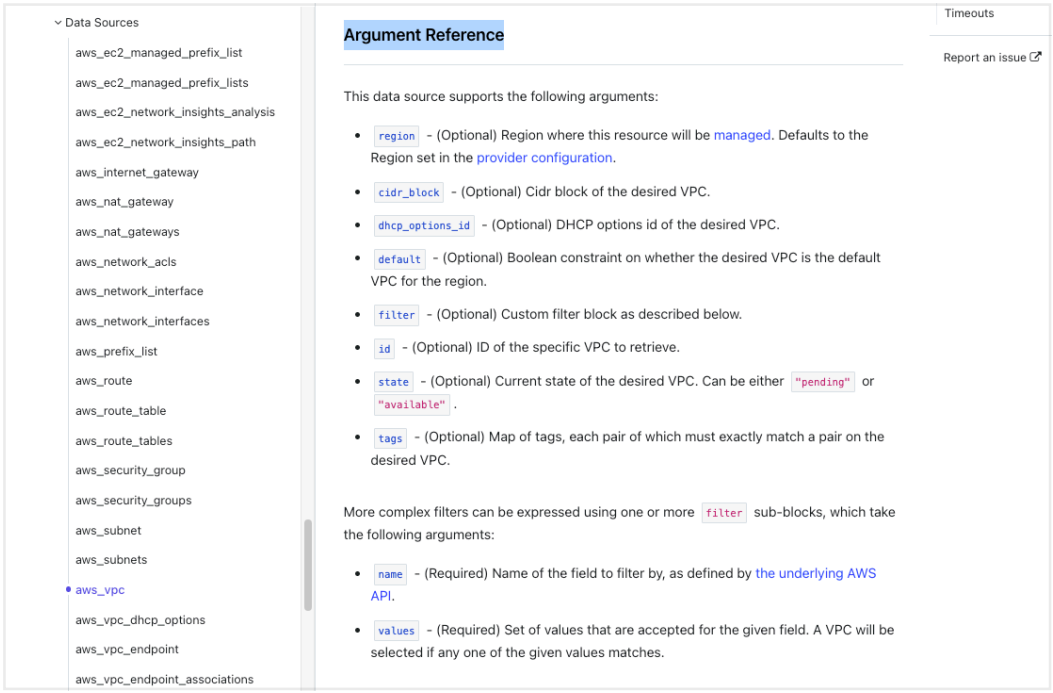


And in the block we add arguments which are the filter for the query

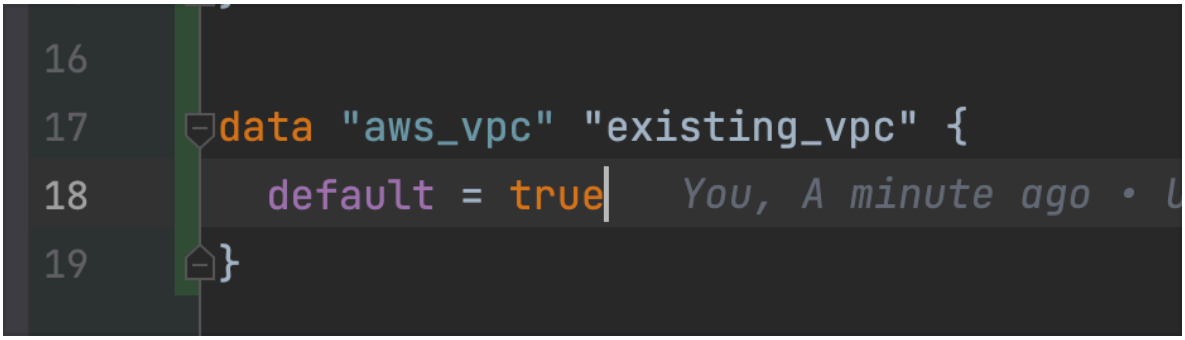
Here is the documentation for data source -> aws_vpc



And here are the argument references we can use for data source -> aws_vpc



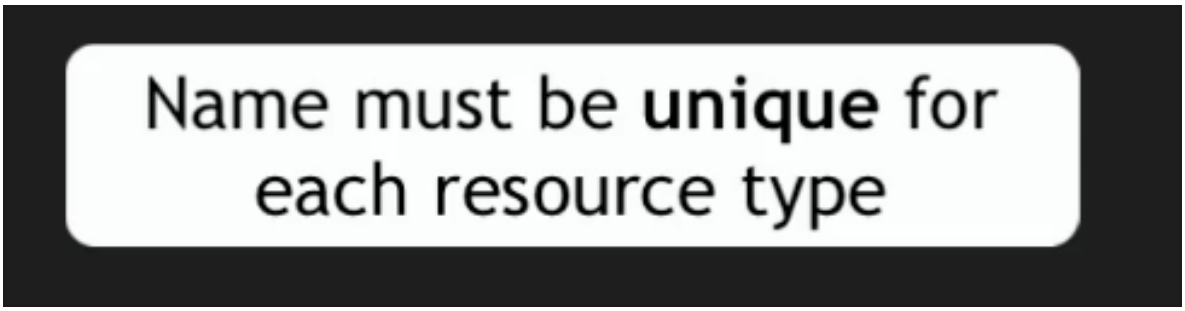
We want terraform to give us an AWS default VPC



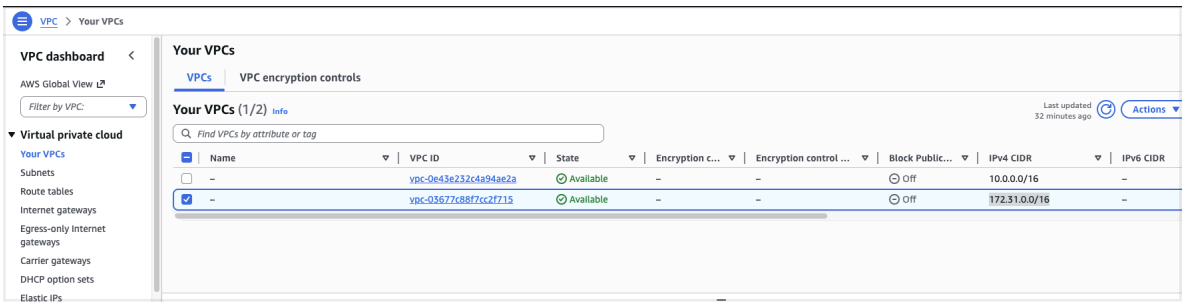
And we want to create a subnet in that existing VPC, but this new resource will reference the ‘data source’ object instead of a ‘resource’ object:

```
16
17  data "aws_vpc" "existing_vpc" {
18    default = true
19  }
20
21  resource "aws_subnet" "dev-subnet-2" {
22    vpc_id = data.aws_vpc.existing_vpc.id    You, A
23    cidr_block = "10.0.10.0/24"
24    availability_zone = "eu-west-2a"
25  }
```

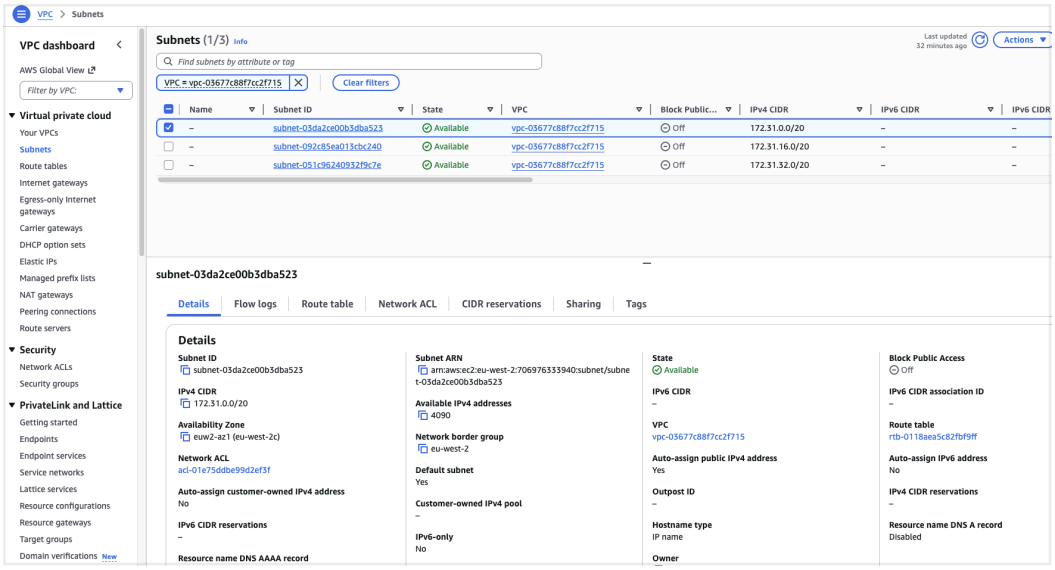
The subnet name is ‘dev-subnet-2’ because each name must be unique for each resource type.



The code needs another change, the CIDR block must be a section of the default VPC’s IP address range: 172.31.0.0/16



And the default VPC already have subnets that were created by default:

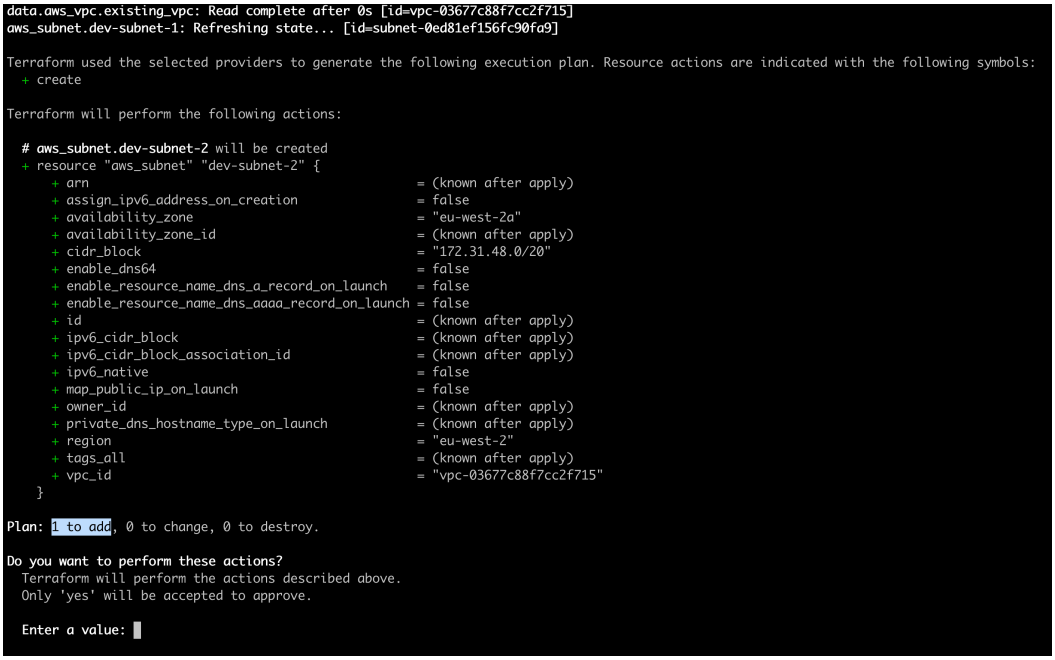


And we need each subnet to have a different set of IP addresses. So we need to choose a range that is not assigned to the existing subnets.

So I will take the last range: 172.31.32.0/20 and from that we take the next subset -> 172.31.48.0/20

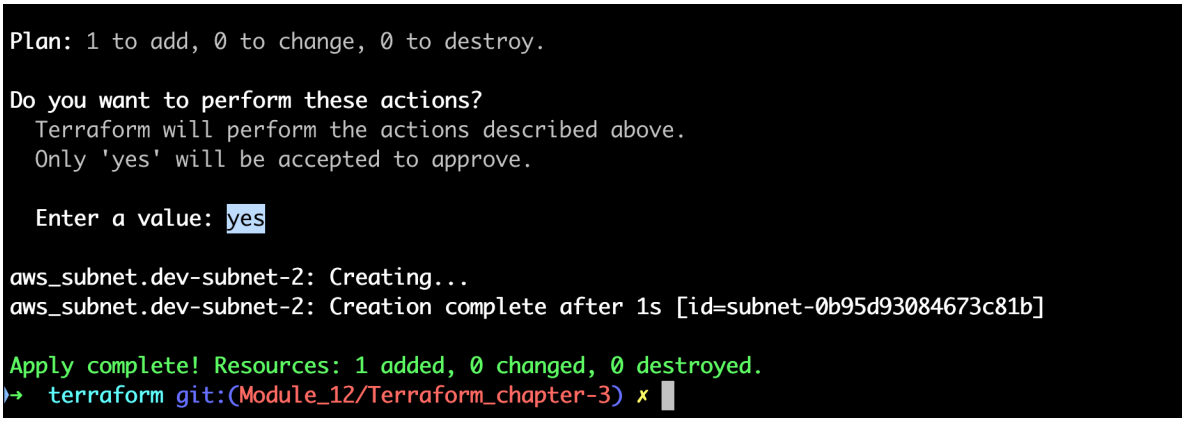
```
17  data "aws_vpc" "existing_vpc" {
18    default = true
19  }
20
21  resource "aws_subnet" "dev-subnet-2" {
22    vpc_id = data.aws_vpc.existing_vpc.id
23    cidr_block = "172.31.48.0/20"    You, Moments ago •
24    availability_zone = "eu-west-2a"
25  }
```

-> terraform apply

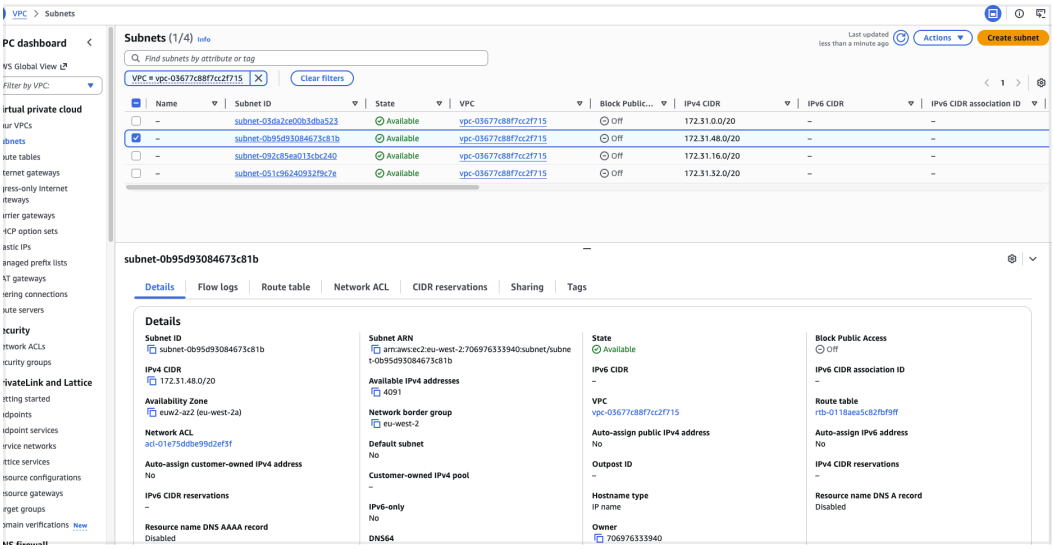


And tf calculates the difference and the actions that it needs to take in order to fulfill the desired state. So the plan shows that it will create 1 resource - ‘dev-subnet-2′ with all the attributes that are listed.

Now we confirm the plan -> yes

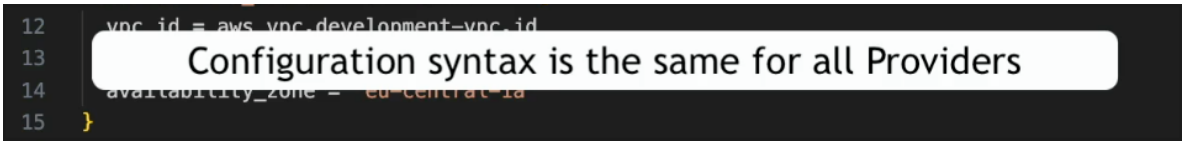


It is done! Let’s check in the UI



RECAP

Consistent syntax



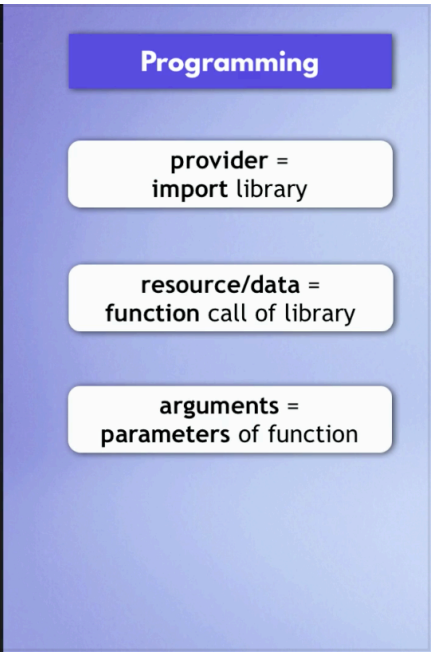
Provider -> is like import library

Resource / data -> is like a function call of a library
- resource -> function that creates something
- data -> function that returns something that already exi

Arguments -> are like the parameters of a function

main.tf providers.tf

```hcl
provider "aws" {
  region = "eu-central-1"
  access_key = "AKIAUZ7NN5BBID2IQDHY"
  secret_key = "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"
}

resource "aws_vpc" "development-vpc" {
  cidr_block = "10.0.0.0/16"
}

resource "aws_subnet" "dev-subnet-1" {
  vpc_id = aws_vpc.development-vpc.id
  cidr_block = "10.0.10.0/24"
  availability_zone = "eu-central-1a"
}

data "aws_vpc" "existing_vpc" {
  default = true
}

resource "aws_subnet" "dev-subnet-2" {
  vpc_id = data.aws_vpc.existing_vpc.id
  cidr_block = "172.31.48.0/20"
  availability_zone = "eu-central-1a"
}
```



We need credentials:

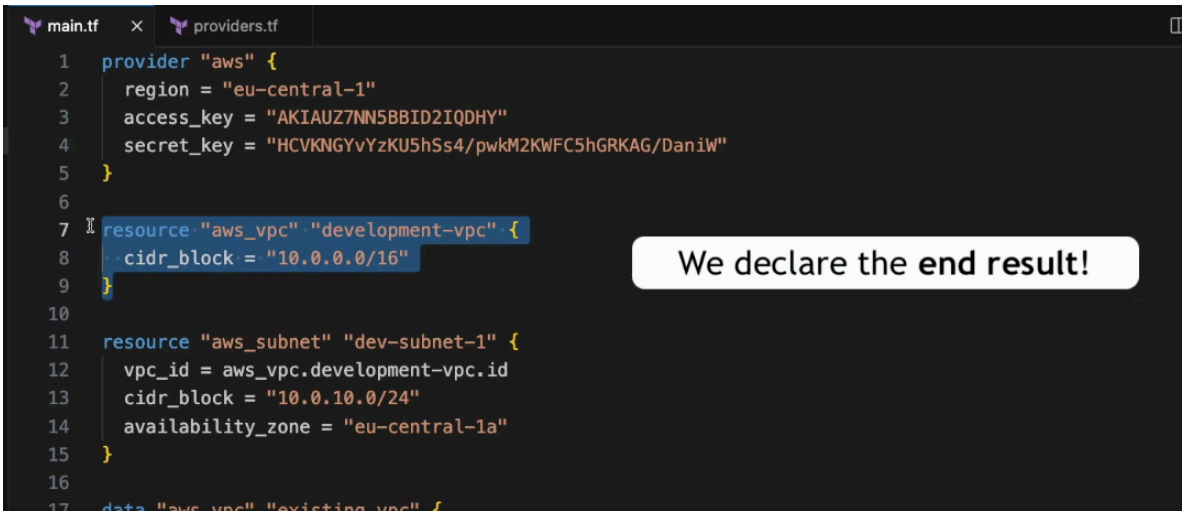
```hcl
  region = "eu-central-1"
  access_key = "AKIAUZ7NN5BBID2IQDHY"
  secret_key = "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"
}

resource "aws_vpc" "development-vpc" {
  cidr_block = "10.0.0.0/16"
}

resource "aws_subnet" "dev-subnet-1" {
  vpc_id = aws_vpc.development-vpc.id
  cidr_block = "10.0.10.0/24"
  availability_zone = "eu-central-1a"
}

data "aws_vpc" "existing_vpc" {
  default = true
```

What happens if I execute
-> terraform apply

```
→  terraform git:(Module_12/Terraform_chapter-3) ✗ terraform apply
data.aws_vpc.existing_vpc: Reading...
aws_vpc.development-vpc: Refreshing state... [id=vpc-0e43e232c4a94ae2a]
data.aws_vpc.existing_vpc: Read complete after 0s [id=vpc-03677c88f7cc2f715]
aws_subnet.dev-subnet-2: Refreshing state... [id=subnet-0b95d93084673c81b]
aws_subnet.dev-subnet-1: Refreshing state... [id=subnet-0ed81ef156fc90fa9]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
→  terraform git:(Module_12/Terraform_chapter-3) ✗
```

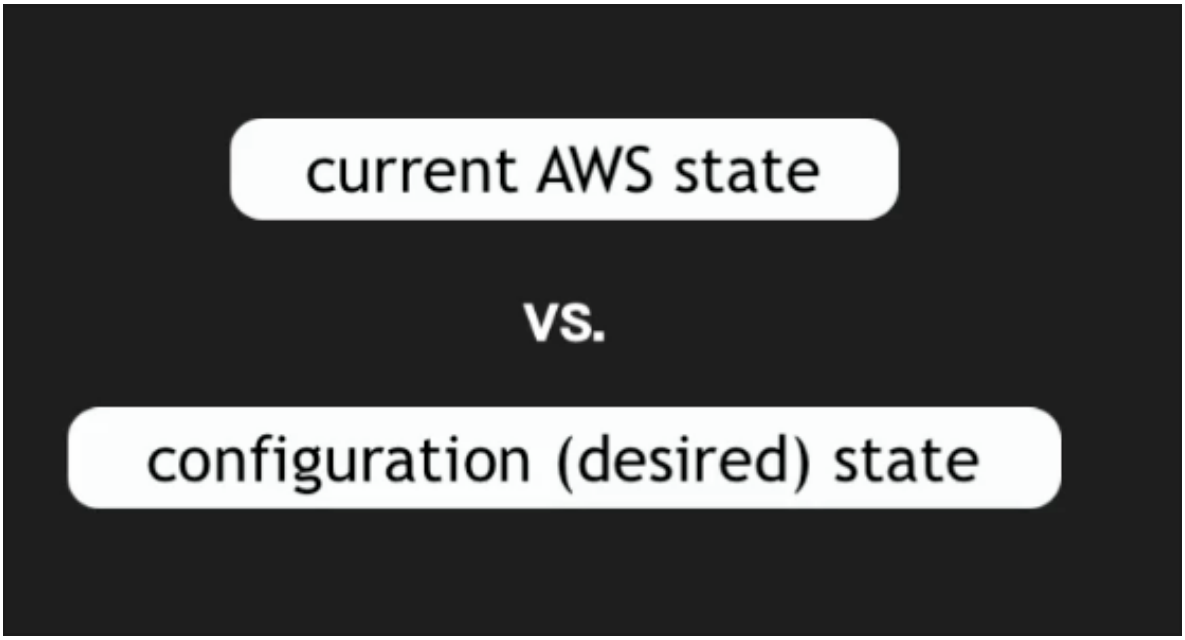
Tf refreshed the state for each resource and figured out that no changes are need it.

TF declare the end result, we didn’t say how



Then when we -> terraform apply

TF figures out the current AWS state and compares it with the desired state



If the states differed then TF creates whatever needs to be created.

This gives TF idempotency -> if we apply the TF config 100 times, then 100 times the result is the same.

Another advantage is that the user doesn’t need to remember the current state.

main.tf providers.tf

```hcl
provider "aws" {
  region = "eu-central-1"
  access_key = "AKIAUZ7NN5BBID2IQDHY"
  secret_key = "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"
}

resource "aws_vpc" "development-vpc" {
  cidr_block = "10.0.0.0/16"
}

resource "aws_subnet" "dev-subnet-1" {
  vpc_id = aws_vpc.development-vpc.id
  cidr_block = "10.0.10.0/24"
  availability_zone = "eu-central-1a"
}

data "aws_vpc" "existing_vpc" {
  default = true
}

resource "aws_subnet" "dev-subnet-2" {
  vpc_id = data.aws_vpc.existing_vpc.id
  cidr_block = "172.31.48.0/20"
  availability_zone = "eu-central-1a"
}
```

Idempodent

No need to remember the current state

TECHWORLD WITH NANA